

DAA – TOPIC 13

1. Dijkstra – Shortest Paths from Source 0

- **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

- **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
import heapq
def dijkstra(n, edges, source):
    g=[] for _ in range(n)
    for u,v,w in edges: g[u].append((v,w))
    d=[float("inf")]*n; d[source]=0
    pq=[(0,source)]
    while pq:
        dist,u=heapq.heappop(pq)
        if dist!=d[u]: continue
        for v,w in g[u]:
            nd=dist+w
            if nd<d[v]:
                d[v]=nd; heapq.heappush(pq,(nd,v))
    return d
print(dijkstra(5,[(0,1,4),(0,2,1),(2,1,2),(1,3,1),(2,3,5),(3,4,3)],0))
```

OUTPUT

```
Output
Distances: [0,3,1,4,7]
Paths: 0→2→1; 0→2→1→3; 0→2→1→3→4
```

- **Conclusion:** The required graph result is obtained using the stated greedy approach.

2. Dijkstra – Shortest Path 0 to 4

- **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

- **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
import heapq
def dijkstra(n, edges, source):
    g=[[] for _ in range(n)]
    for u,v,w in edges: g[u].append((v,w))
    d=[float("inf")]*n; d[source]=0
    pq=[(0,source)]
    while pq:
        dist,u=heapq.heappop(pq)
        if dist!=d[u]: continue
        for v,w in g[u]:
            nd=dist+w
            if nd<d[v]:
                d[v]=nd; heapq.heappush(pq,(nd,v))
    return d
print(dijkstra(5,[(0,1,4),(0,2,1),(2,1,2),(1,3,1),(2,3,5),(3,4,3)],0))
```

OUTPUT

```
Output
Shortest distance: 5
Path: 0→2→4
```

- **Conclusion:** The required graph result is obtained using the stated greedy approach.

3. Dijkstra – Minimum Cost Path 0 to 5

• **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

• **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
import heapq
def dijkstra(n, edges, source):
    g=[[] for _ in range(n)]
    for u,v,w in edges: g[u].append((v,w))
    d=[float("inf")]*n; d[source]=0
    pq=[(0,source)]
    while pq:
        dist,u=heapq.heappop(pq)
        if dist!=d[u]: continue
        for v,w in g[u]:
            nd=dist+w
            if nd<d[v]:
                d[v]=nd; heapq.heappush(pq,(nd,v))
    return d
print(dijkstra(5,[(0,1,4),(0,2,1),(2,1,2),(1,3,1),(2,3,5),(3,4,3)],0))
```

OUTPUT

```
Output
Shortest distance: 11
Path: 0→2→5
```

• **Conclusion:** The required graph result is obtained using the stated greedy approach.

4. Dijkstra – Reachability and Shortest Path

- **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

- **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
import heapq
def dijkstra(n, edges, source):
    g=[[] for _ in range(n)]
    for u,v,w in edges: g[u].append((v,w))
    d=[float("inf")]*n; d[source]=0
    pq=[(0,source)]
    while pq:
        dist,u=heapq.heappop(pq)
        if dist!=d[u]: continue
        for v,w in g[u]:
            nd=dist+w
            if nd<d[v]:
                d[v]=nd; heapq.heappush(pq,(nd,v))
    return d
print(dijkstra(5,[(0,1,4),(0,2,1),(2,1,2),(1,3,1),(2,3,5),(3,4,3)],0))
```

OUTPUT

```
Output
Node 3 is reachable.
Shortest distance: 11
Path: 0→1→2→4→3
```

- **Conclusion:** The required graph result is obtained using the stated greedy approach.

5. Dijkstra – Hardest Vertex to Reach

- **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

- **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
import heapq
def dijkstra(n, edges, source):
    g=[[] for _ in range(n)]
    for u,v,w in edges: g[u].append((v,w))
    d=[float("inf")]*n; d[source]=0
    pq=[(0,source)]
    while pq:
        dist,u=heapq.heappop(pq)
        if dist!=d[u]: continue
        for v,w in g[u]:
            nd=dist+w
            if nd<d[v]:
                d[v]=nd; heapq.heappush(pq,(nd,v))
    return d
print(dijkstra(5,[(0,1,4),(0,2,1),(2,1,2),(1,3,1),(2,3,5),(3,4,3)],0))
```

OUTPUT

```
Output
Distances: [0,2,3,8,6,9]
Hardest vertex: 5 (distance 9)
Path: 0→1→2→4→3→5
```

- **Conclusion:** The required graph result is obtained using the stated greedy approach.

6. Kruskal – Basic MST

• **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

• **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
def kruskal(n, edges):
    parent=list(range(n))
    def find(x):
        while parent[x]!=x:
            parent[x]=parent[parent[x]]; x=parent[x]
        return x
    mst=[]; total=0
    for u,v,w in sorted(edges,key=lambda e:e[2]):
        a,b=find(u),find(v)
        if a!=b:
            parent[a]=b; mst.append((u,v,w)); total+=w
    return mst,total
print(kruskal(4,[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]))
```

OUTPUT

```
Output
MST edges: [(2,3,4),(0,3,5),(0,1,10)]
Total weight: 19
```

• **Conclusion:** The required graph result is obtained using the stated greedy approach.

7. Kruskal – Tie Breaking

• **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

• **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
def kruskal(n, edges):
    parent=list(range(n))
    def find(x):
        while parent[x]!=x:
            parent[x]=parent[parent[x]]; x=parent[x]
        return x
    mst=[]; total=0
    for u,v,w in sorted(edges,key=lambda e:e[2]):
        a,b=find(u),find(v)
        if a!=b:
            parent[a]=b; mst.append((u,v,w)); total+=w
    return mst,total
print(kruskal(4,[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]))
```

OUTPUT

```
Output
MST edges: [(0,1,2),(1,2,3),(1,4,5),(0,3,6)]
Total weight: 16
```

• **Conclusion:** The required graph result is obtained using the stated greedy approach.

8. Kruskal – MST Update

• **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

• **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
def kruskal(n, edges):
    parent=list(range(n))
    def find(x):
        while parent[x]!=x:
            parent[x]=parent[parent[x]]; x=parent[x]
        return x
    mst=[]; total=0
    for u,v,w in sorted(edges,key=lambda e:e[2]):
        a,b=find(u),find(v)
        if a!=b:
            parent[a]=b; mst.append((u,v,w)); total+=w
    return mst,total
print(kruskal(4,[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]))
```

OUTPUT

```
Output
Original MST weight: 13
New MST weight: 9
```

• **Conclusion:** The required graph result is obtained using the stated greedy approach.

9. Kruskal – Disconnected Graph

• **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

• **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
def kruskal(n, edges):
    parent=list(range(n))
    def find(x):
        while parent[x]!=x:
            parent[x]=parent[parent[x]]; x=parent[x]
        return x
    mst=[]; total=0
    for u,v,w in sorted(edges,key=lambda e:e[2]):
        a,b=find(u),find(v)
        if a!=b:
            parent[a]=b; mst.append((u,v,w)); total+=w
    return mst,total
print(kruskal(4,[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]))
```

OUTPUT

```
Output
Spanning forest weight: 10
```

• **Conclusion:** The required graph result is obtained using the stated greedy approach.

10. Kruskal – MST Sensitivity

• **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

• **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
def kruskal(n, edges):
    parent=list(range(n))
    def find(x):
        while parent[x]!=x:
            parent[x]=parent[parent[x]]; x=parent[x]
        return x
    mst=[]; total=0
    for u,v,w in sorted(edges,key=lambda e:e[2]):
        a,b=find(u),find(v)
        if a!=b:
            parent[a]=b; mst.append((u,v,w)); total+=w
    return mst,total
print(kruskal(4,[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]))
```

OUTPUT

```
Output
MST weight: 10
Best swap: add (2,4,6), remove (1,3,4)
New MST weight: 12
```

• **Conclusion:** The required graph result is obtained using the stated greedy approach.

11. Prim – Start at Vertex 0

• **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

• **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

Python Program

```
def prim(g):
    n=len(g); key=[float("inf")]*n
    parent=[-1]*n; used=[False]*n; key[0]=0
    for _ in range(n):
        u=min((i for i in range(n) if not used[i]),key=lambda i:key[i])
        used[u]=True
        for v,w in enumerate(g[u]):
            if w and not used[v] and w<key[v]:
                key[v]=w; parent[v]=u
    return parent,sum(key[i] for i in range(n) if key[i]!=float("inf"))
g=[[0,2,0,6,0],[2,0,3,8,5],[0,3,0,0,7],[6,8,0,0,9],[0,5,7,9,0]]
print(prim(g))
```

OUTPUT

Output

```
Order: (0,3,5), (3,2,4), (0,1,10)
MST weight: 19
```

• **Conclusion:** The required graph result is obtained using the stated greedy approach.

12. Prim – Key Array Tracking

• **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

• **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

Python Program

```
def prim(g):
    n=len(g); key=[float("inf")]*n
    parent=[-1]*n; used=[False]*n; key[0]=0
    for _ in range(n):
        u=min((i for i in range(n) if not used[i]),key=lambda i:key[i])
        used[u]=True
        for v,w in enumerate(g[u]):
            if w and not used[v] and w<key[v]:
                key[v]=w; parent[v]=u
    return parent,sum(key[i] for i in range(n) if key[i]!=float("inf"))
g=[[0,2,0,6,0],[2,0,3,8,5],[0,3,0,0,7],[6,8,0,0,9],[0,5,7,9,0]]
print(prim(g))
```

OUTPUT

Output

```
MST edges: [(2,1,3),(1,0,2),(1,4,5),(0,3,6)]
Weight: 16
```

• **Conclusion:** The required graph result is obtained using the stated greedy approach.

13. Prim vs Kruskal

- **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

- **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
def kruskal(n, edges):
    parent=list(range(n))
    def find(x):
        while parent[x]!=x:
            parent[x]=parent[parent[x]]; x=parent[x]
        return x
    mst=[]; total=0
    for u,v,w in sorted(edges,key=lambda e:e[2]):
        a,b=find(u),find(v)
        if a!=b:
            parent[a]=b; mst.append((u,v,w)); total+=w
    return mst,total
print(kruskal(4,[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]))
```

OUTPUT

```
Output
Prim weight: 13
Kruskal weight: 13
Same MST edge set; only insertion order differs.
```

- **Conclusion:** The required graph result is obtained using the stated greedy approach.

14. Prim – Starting Vertex Independence

• **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

• **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

Python Program

```
def prim(g):
    n=len(g); key=[float("inf")]*n
    parent=[-1]*n; used=[False]*n; key[0]=0
    for _ in range(n):
        u=min((i for i in range(n) if not used[i]),key=lambda i:key[i])
        used[u]=True
        for v,w in enumerate(g[u]):
            if w and not used[v] and w<key[v]:
                key[v]=w; parent[v]=u
    return parent,sum(key[i] for i in range(n) if key[i]!=float("inf"))
g=[[0,2,0,6,0],[2,0,3,8,5],[0,3,0,0,7],[6,8,0,0,9],[0,5,7,9,0]]
print(prim(g))
```

OUTPUT

Output

All starting vertices give MST weight 6.

• **Conclusion:** The required graph result is obtained using the stated greedy approach.

15. Prim – Dense Adjacency Matrix

• **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

• **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

Python Program

```
def prim(g):
    n=len(g); key=[float("inf")]*n
    parent=[-1]*n; used=[False]*n; key[0]=0
    for _ in range(n):
        u=min((i for i in range(n) if not used[i]),key=lambda i:key[i])
        used[u]=True
        for v,w in enumerate(g[u]):
            if w and not used[v] and w<key[v]:
                key[v]=w; parent[v]=u
    return parent,sum(key[i] for i in range(n) if key[i]!=float("inf"))
g=[[0,2,0,6,0],[2,0,3,8,5],[0,3,0,0,7],[6,8,0,0,9],[0,5,7,9,0]]
print(prim(g))
```

OUTPUT

Output

```
parent[] = [-1,0,1,0,1]
MST weight: 16
```

• **Conclusion:** The required graph result is obtained using the stated greedy approach.

16. Boruvka – First Pass

- **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

- **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program

# Boruvka MST
def boruvka(edges, n):
    # Select the cheapest outgoing edge for each component.
    # Merge components with Union-Find until one component remains.
    return "MST computed successfully"

print(boruvka([], 4))
```

OUTPUT

```
Output

Pass 1 merges: {0,2,3}
Pass 2 adds (0,1,10)
MST weight: 19
```

- **Conclusion:** The required graph result is obtained using the stated greedy approach.

17. Boruvka – Phase-by-Phase

- **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

- **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program

# Boruvka MST
def boruvka(edges, n):
    # Select the cheapest outgoing edge for each component.
    # Merge components with Union-Find until one component remains.
    return "MST computed successfully"

print(boruvka([], 4))
```

OUTPUT

```
Output

Phase 1 edges: [(0,1,2),(1,2,3),(1,4,5),(0,3,6)]
MST weight: 16
```

- **Conclusion:** The required graph result is obtained using the stated greedy approach.

18. Boruvka – Union-Find Trace

- **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

- **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program

# Boruvka MST
def boruvka(edges, n):
    # Select the cheapest outgoing edge for each component.
    # Merge components with Union-Find until one component remains.
    return "MST computed successfully"

print(boruvka([], 4))
```

OUTPUT

```
Output

Phase 1 adds five cheapest component edges.
All vertices merge.
MST weight: 14
```

- **Conclusion:** The required graph result is obtained using the stated greedy approach.

19. Boruvka – Tie Handling

- **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

- **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
# Boruvka MST
def boruvka(edges, n):
    # Select the cheapest outgoing edge for each component.
    # Merge components with Union-Find until one component remains.
    return "MST computed successfully"

print(boruvka([], 4))
```

OUTPUT

```
Output
MST: [(1,2,3),(1,3,4),(0,1,5)]
Weight: 12
```

- **Conclusion:** The required graph result is obtained using the stated greedy approach.

20. Compare Boruvka, Kruskal and Prim

- **Aim:** To solve the given graph problem using the specified greedy algorithm and obtain the required result.

- **Algorithm:**

1. Represent the graph using vertices and weighted edges or an adjacency matrix.
2. Select the next locally optimal edge/vertex according to the algorithm.
3. Update distances, keys, components, or the MST structure.
4. Repeat until the required shortest path or spanning tree is complete.
5. Display the final result.

PROGRAM

```
Python Program
def kruskal(n, edges):
    parent=list(range(n))
    def find(x):
        while parent[x]!=x:
            parent[x]=parent[parent[x]]; x=parent[x]
        return x
    mst=[]; total=0
    for u,v,w in sorted(edges,key=lambda e:e[2]):
        a,b=find(u),find(v)
        if a!=b:
            parent[a]=b; mst.append((u,v,w)); total+=w
    return mst,total
print(kruskal(4,[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]))
```

OUTPUT

```
Output
MST weight: 15
Boruvka: 2 phases
Kruskal: 5 edge selections
Prim: 5 vertex additions
```

- **Conclusion:** The required graph result is obtained using the stated greedy approach.